The Tools We'll Use

Over the course of the book, we'll walk through the basics of getting Rails set up to serve JavaScript and CSS to the browser. Then we will write code to get the browser to do things. We're going to look at two different frameworks that have very different approaches—Hotwire and React:

- Hotwire is a framework that allows you to keep most of your logic on the server and communicate with the client by sending HTML.[1] Much of the Hotwire code uses Turbo, which is a library that allows you to do complex client-server interactions without writing custom JavaScript. Turbo itself consists of Turbo Drive, which is the successor to Turbolinks and allows you to speed up basic links through your site; Turbo Frames, which allows you to easily replace part of your page with new HTML from the server; and Turbo Streams, which allows you to do arbitrary Document Object Model (DOM) replacement without any custom JavaScript. Hotwire also includes Stimulus, a JavaScript library that manages client-side interactions more directly.

- React is a framework where most of the rendering logic is on the client.[2] In React, you describe your output using JSX, a language for specifying HTML in JavaScript. You also describe what variables make up the state of the system, and when that state changes, React automatically redraws the parts of the screen that reflect the new state. React typically communicates with the server as an API and frequently expects to receive JSON data in return, which is used to update the state.

  We will use other more foundational tools—TypeScript, esbuild, jsbundling, and cssbundling—to build the infrastructure of our application, no matter what JavaScript frameworks we use on top:

- TypeScript is an extension of JavaScript that provides type checking and type inference, which means TypeScript ensures that values in your code have the types you expect.[3] It's a superset of JavaScript, which means that